ASDD PRACTICAL EXAM

Theory Questions and Answers – All 7 Experiments

Simple, exam-ready answers for practical viva and journal writing

This document covers every theory question included in the ASDD practical experiment list provided.

Experiment 1: Jira Board and Sprint Management

Q1. Differentiate between Product Backlog and Sprint Backlog.

Product Backlog	Sprint Backlog
Contains all features, requirements, improvements and bug fixes needed for the product.	Contains the Product Backlog items selected for the current sprint and the plan to complete them.
Maintained and ordered by the Product Owner.	Created and managed by the Developers during Sprint Planning.
Includes current and future work.	Focuses on the work selected for one sprint.
Can be updated as the product and requirements change.	Can be updated by Developers as they learn more during the sprint.
Used for overall product planning.	Used for planning and tracking work during the sprint.

Example: For a Student Management System, the Product Backlog may contain Login, Registration, Student Records, Attendance and Results. If Login, Registration and Student Records are selected for the next sprint, they become part of that Sprint Backlog.

Q2. Define Product Backlog and Sprint Backlog.

Product Backlog: The Product Backlog is a prioritized list of all features, requirements, improvements and bug fixes that may be needed in a software product. It is maintained by the Product Owner and is continuously refined.

- Contains all possible product work.
- Items are ordered according to priority and value.
- It changes as new requirements and feedback are received.

Sprint Backlog: The Sprint Backlog is the set of Product Backlog items selected for a particular sprint, together with the plan for delivering them. It is managed by the Developers.

- Created during Sprint Planning.
- Contains the work planned for the current sprint.
- Helps the team track and complete sprint work.

Q3. How is the Sprint Process carried out in an Agile Team?

A Sprint is a fixed-length period in Scrum during which the team works to achieve a Sprint Goal and produce a usable product increment.

1. Sprint Planning: The team discusses the product priorities, selects work from the Product Backlog and defines a Sprint Goal.
2. Sprint Backlog Creation: Selected backlog items are broken into smaller tasks and a plan is prepared.
3. Sprint Execution: Developers work on the selected tasks during the sprint.
4. Daily Scrum: Developers hold a short daily meeting to inspect progress and plan the next day's work.

5. Development and Testing: Features are developed and checked against the acceptance criteria.
6. Sprint Review: The team demonstrates the completed work to stakeholders and gathers feedback.
7. Sprint Retrospective: The team discusses what went well, what problems occurred and what can be improved.
8. Next Sprint: The Product Backlog is updated and the team begins planning the next sprint.

Example: A two-week sprint for a Student Management System may focus on implementing student registration and login.

Experiment 2: Jira and GitHub Integration

Q1. How can Jira issues be linked to GitHub branches and commits? How is it useful for others to track changes?

Jira issues can be linked to GitHub branches, commits and pull requests by connecting a GitHub repository to Jira using the GitHub for Jira integration.

9. Create a Story or Task in Jira and note its issue key, for example ABC-12.
10. Connect the GitHub repository to Jira using the GitHub for Jira integration. This normally requires an authorized Jira administrator to install and configure the integration.
11. Create a branch whose name includes the Jira issue key, for example:

```
git checkout -b feature/ABC-12-login
```

4. Make the required code changes, stage them and commit with the Jira key in the commit message:

```
git add .
git commit -m "ABC-12: Add login feature"
```

5. Push the branch to GitHub:

```
git push -u origin feature/ABC-12-login
```

12. Open the Jira issue and check its Development section for the linked branch, commit or pull request.
13. Create a pull request if required and include the Jira issue key in its title or description.

Usefulness:

- Easy tracking: Team members can identify which code changes belong to a Jira issue.
- Collaboration: Developers can understand the work done by other team members.
- Transparency: Project managers can track development progress.
- Traceability: Code changes can be connected to requirements and bugs.
- Debugging: Developers can locate commits related to a reported issue.

Example: If ABC-12 represents a login feature, branches and commits containing ABC-12 can be associated with that Jira issue when the integration is correctly configured.

Important: Using the Jira key in a branch or commit name alone does not guarantee that Jira will display development information. The GitHub integration must be installed, authorized and connected to the repository.

Experiment 3: Git Commands and Version Control

Q1. Explain the purpose of git add, git commit, git push, git branch and git status.

1. git add

Adds new or modified files to the staging area. It prepares changes for the next commit.

```
git add .
```

2. git commit

Saves the staged changes as a snapshot in the local Git repository. A commit message describes the changes.

```
git commit -m "Added login page"
```

3. git push

Uploads local commits to a remote repository such as GitHub so that others can access the changes.

```
git push
```

4. git branch

Lists, creates or manages branches. Branches allow developers to work on features independently.

```
git branch
```

5. git status

Displays the current state of the working directory and staging area, including modified, untracked and staged files.

```
git status
```

Simple Git workflow:

```
Modify files  →  git add  →  git commit  →  git push
```

Q2. Explain GitHub Actions.

GitHub Actions is an automation platform provided by GitHub. It is used to automatically build, test and deploy applications when events such as code pushes or pull requests occur.

Important components:

- Workflow: A set of automated tasks defined in a YAML file.
- Event: An activity that triggers a workflow, such as push or pull_request.
- Job: A group of steps executed on a runner.
- Step: An individual task inside a job.
- Runner: A machine that executes the workflow.
- Action: A reusable unit of code that performs a task.

Working:

14. A developer pushes code to GitHub.
15. The configured event triggers the workflow.
16. GitHub starts a runner and checks out the source code.
17. The application is built and automated tests are run.
18. If the steps succeed, the workflow can deploy the application.

Example YAML workflow:

```
name: Node CI

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm install
```

```
      - name: Run tests
        run: npm test
```

Advantages:

- Automates build, testing and deployment.
- Reduces manual work.
- Detects errors early.
- Supports continuous integration and continuous delivery/deployment.
- Works directly with GitHub repositories.

Experiment 4: Docker Image Management and Container Execution

Q1. Differentiate between a Docker image and a Docker container.

Docker Image	Docker Container
A read-only template used to create containers.	A running or stopped instance of an image.
Contains application code, libraries and dependencies.	Provides an environment in which the application executes.
Created using a Dockerfile or downloaded from a registry.	Created using the docker run command.
Acts as the template or blueprint.	Acts as the instance created from the template.
One image can be used to create many containers.	Each container is an individual instance.

Example: nginx is a Docker image. A web server started from that image is a Docker container.

Q2. Explain the Docker build command.

The docker build command creates a Docker image using instructions written in a Dockerfile and the files in the build context.

Syntax:

```
docker build -t image_name .
```

- docker build: Starts the image-building process.
- -t: Assigns a name or tag to the image.
- image_name: The name given to the new image.
- .: Uses the current directory as the build context.

Steps:

19. Create the application files.
20. Create a Dockerfile with the required instructions.
21. Open the terminal in the project directory.
22. Run docker build -t myapp .
23. Docker reads the Dockerfile and executes its instructions.
24. Verify the created image using docker images.

Output: A Docker image named myapp is created and can be used to start containers.

Experiment 5: Dockerfile Creation and Image Building

Q1. What is a Dockerfile? Explain FROM, COPY, RUN, EXPOSE and CMD.

A Dockerfile is a text file containing instructions used to build a Docker image. It defines the base environment, application files, dependencies and default startup command.

FROM

Specifies the base image used to build the new image. It is generally the first instruction.

```
FROM node:20
```

COPY

Copies files or directories from the build context into the image.

```
COPY . /app
```

RUN

Executes commands during image building, commonly to install dependencies.

```
RUN npm install
```

EXPOSE

Documents the port on which the application listens inside the container. It does not publish the port to the host by itself.

```
EXPOSE 3000
```

CMD

Specifies the default command to run when a container starts. It can be overridden when running the container.

```
CMD ["npm", "start"]
```

Complete example:

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Working:

25. Select a base image using FROM.
26. Set the working directory.
27. Copy application files using COPY.
28. Install dependencies using RUN.
29. Document the application port using EXPOSE.
30. Specify the default startup command using CMD.
31. Build the Docker image using docker build.

Advantages: Dockerfiles automate image creation, provide a consistent environment, reduce manual configuration and make applications easier to deploy.

Experiment 6: Terraform and GitHub Actions

Q1. What is Terraform? List the Terraform commands and explain their purpose briefly.

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It is used to create, manage and provision infrastructure such as virtual machines, networks and databases through configuration files.

- Automates infrastructure creation and management.
- Supports cloud platforms such as AWS, Azure and Google Cloud.
- Uses HCL (HashiCorp Configuration Language).
- Maintains a state file to track managed resources.
- Helps create repeatable and consistent infrastructure.

Command	Purpose
terraform init	Initializes the working directory and downloads required providers.
terraform validate	Checks whether the configuration is valid.
terraform fmt	Formats Terraform configuration files.
terraform plan	Previews the changes Terraform intends to make.
terraform apply	Creates or updates the infrastructure.
terraform show	Displays the current state or a saved plan.
terraform output	Displays output values defined in the configuration.
terraform destroy	Destroys infrastructure managed by the configuration.
terraform state list	Lists resources recorded in the state file.

Working:

32. Write infrastructure configuration in a .tf file.
33. Run terraform init.
34. Run terraform validate.
35. Run terraform plan to preview changes.
36. Run terraform apply to create or update resources.
37. Verify the resources.
38. Run terraform destroy when test infrastructure is no longer needed.

Example Terraform configuration:

```
resource "local_file" "example" {
  filename = "hello.txt"
  content  = "Hello from Terraform"
}
```

This example uses the local provider to create a file named hello.txt.

Advantages: Saves time, reduces manual errors, supports repeatable infrastructure, tracks infrastructure changes and works with multiple cloud providers.

Experiment 7: Grafana Using Docker

Q1. What is Grafana, and how is it used for monitoring and visualization in DevOps?

Grafana is an open-source monitoring and data visualization platform. It displays data from different sources in dashboards, graphs, charts and tables.

Uses in DevOps:

- Monitoring: Tracks application, server and infrastructure performance.
- Visualization: Displays metrics using dashboards, graphs and charts.
- Performance tracking: Helps monitor CPU, memory, disk and network usage.
- Alerting: Sends alerts when monitored values cross defined limits.
- Data integration: Connects to data sources such as Prometheus, InfluxDB and Elasticsearch.
- Troubleshooting: Helps identify performance issues and system failures.

How Grafana works:

39. A monitoring tool such as Prometheus collects system metrics.
40. Grafana connects to Prometheus as a data source.
41. Grafana queries the collected data.
42. It displays the data in dashboards and visualizations.
43. Users monitor the system and configure alerts.

Examples of metrics monitored:

- CPU utilization.
- RAM consumption.
- Disk usage.
- Network traffic.
- Application response time.
- Server availability.

Advantages:

- Provides interactive dashboards.
- Supports multiple data sources.
- Helps identify performance issues.
- Supports alerts and notifications.
- Improves monitoring and troubleshooting.

Example: Run Grafana using Docker:

```
docker pull grafana/grafana
docker run -d -p 3001:3000 --name grafana grafana/grafana
```

Open http://localhost:3001 in a browser. The host port 3001 is mapped to Grafana's container port 3000.

Quick Revision – One-Line Definitions

Term	Simple meaning
Product Backlog	Prioritized list of all work needed for a product.
Sprint Backlog	Work selected for the current sprint and the plan to complete it.
Sprint	A fixed time period in which a team works to achieve a goal.
Jira–GitHub integration	Connects Jira issues with GitHub branches, commits and pull requests.
Git add	Moves changes to the staging area.
Git commit	Saves staged changes in the local repository.
Git push	Uploads commits to a remote repository.
Git branch	Creates or lists separate lines of development.
Git status	Shows the current state of files in the repository.
GitHub Actions	Automates build, test and deployment workflows.
Docker image	Read-only template used to create containers.
Docker container	An instance of a Docker image.
Dockerfile	Text file containing instructions to build an image.
Terraform	Infrastructure as Code tool used to provision and manage infrastructure.
Grafana	Platform for monitoring and visualizing metrics through dashboards.